UNIT - 5

Diff b/w Branch & Bound & Backtracking:

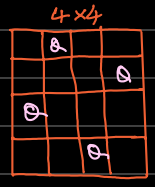
Backtracking	Branch & Bound
⇒ used to find all possible solutions available to a problem. When it realises that it has made a bad choice, it undoes the last choice by backing it up. It searches the state space tree until it has found a soln for the problem.	⇒ When it realises it already has a better optimal soln that the pre-soln leads to, it abandons that pre-soln. It completely searches the state space tree to get optimal soln.
⇒ Backtracking traverses the state space tree by DFS (depth first search) manner.	⇒ It traverses the tree in any manner, DFS or BFS.
⇒ used for solving decision problem	⇒ is used for solving optimisation problem
⇒ more efficient.	⇒ less efficient
⇒ useful for solving N-queen problem, sum of subset.	⇒ useful in solving Knapsack prob, TSP.
⇒ involves a feasibility fun^c.	⇒ involves a bounding fun^c.

Backtracking: → SOSP / → NOP

N- Queen's Problem:

⇒ objective is to place n- queens on a chessboard of dimension n×n such that no two queens attack each other (whr queen can attack row-wise or column-wise or if they are diagonal to each other)

soln



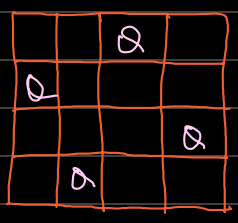
⇒ this can be done using backtracking.

queen: 1 2 3 4 | 2 4 1 3 | col^n

[1st queen in 2nd col, 2nd queen in 4th col, 3rd queen in 1st col ...]

⇒ we can keep the soln in an array eg:

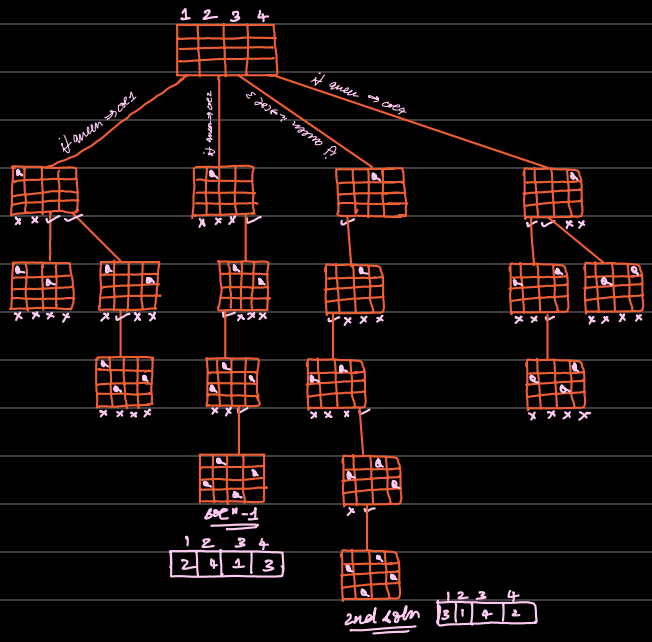
⇒ we can have another soln:



1	2	3	4
3	1	4	2

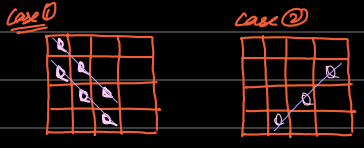
⇒ There are two solns for 4 x 4 N-queen problem.

⇒ What the 1st queen can be placed either in the 1st col, 2nd col, 3rd or 4th.



(We are not getting any soln if we keep 1st queen in col → 1, so we will backtrack and move the position of the 1st queen to 2nd col)

⇒ In the alg it is pretty complex to denote the diagonal attack To get a mathematical eqn to be used in in the alg we can take the below 2 cases:



Coordinates: (1,1)(2,2)(3,3)(4,4)
(2,1)(3,2)(4,3)

Let us consider any 2 points:
(3,2) → (i,j)
(4,3) → (k,l)

$i-j = k-l$
$3-2 = 4-3$
$1 = 1$
$j-l = i-k$

(2,4)(3,3)(4,2)

(3,2) = (i,j)
(4,2) = (k,l)

$i+j = k+l$
$3+3 = 4+2$
$6 = 6$
$j-l = k-i$

$abs(j-l) = abs(k-i)$
absolute

Algorithm NQueens(k,n) → minimal, → no. of queens
// using backtracking the alg prints all possible solns.
for i ← 1 to n do
  if (place(k,i))
    x[k] ← i // place 1st queen in particular col no.
    if k == n
      print x[1...n]
    else

```
else
    NQueens(k+1, n)  // find pos of 2nd queen i.e k+1.
```

```
Algorithm place(k, i)
// returns True if 'i' is placed at kth row and it is common otherwise returns false
for j ← 1 to (k-1) do
    if (x[j] == i) or (abs(x[j] - i) == abs(j - k))
        return false
return True
```

(x[j] == i) → same col; abs(x[j] - i) == abs(j - k) → diagonal attack

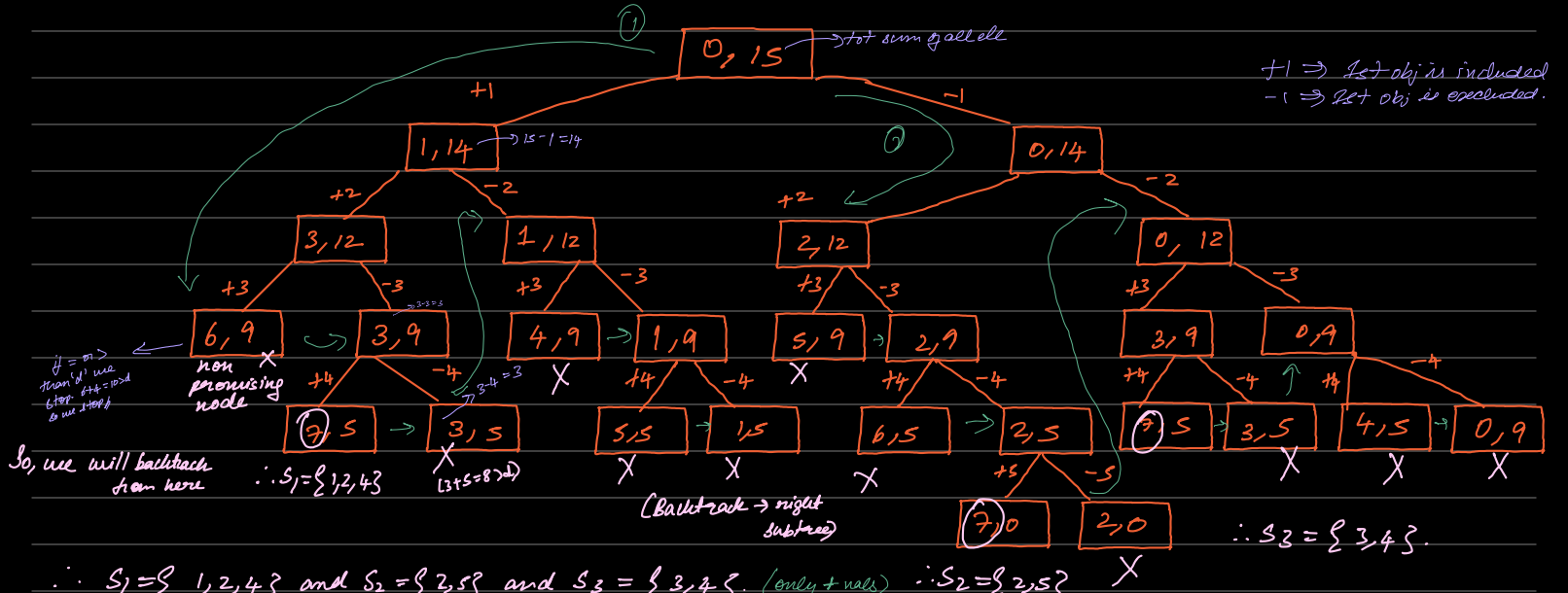
---

Sum of Subset Problem:

⇒ set 'S' will be given with non-negative integers eg: {1, 2, 3, 4, 5} } inputs
⇒ and 'd' will be give that is the max val of subset.
⇒ subsets $S_1, S_2, \ldots$ } output
⇒ The subsets should be derived from the set 'S' such that sum of ele in the subset = d.

→ Should be in increasing order

Say S = {1, 2, 3, 4, 5} and d = 7
first subset ⇒ S_1 = {1, 2, 4} } Sum = 7. solns to sum of subset problem.
⇒ S_2 = {2, 5}
⇒ S_3 = {3, 4}



∴ S_1 = {1, 2, 4} and S_2 = {3, 5} and S_3 = {3, 4}. (only +ve nos) ∴ S_2 = {2, 5}

Time Complexity: $O(2^n)$ (Complete Binary Tree)

```
Algorithm SumOfSub(s, k, r)        // s → initial ⇒ 0
                                   // k → index of ele ⇒ 1
                                   // r → initial is the tot sum of all ele
// I/P: W[1...n] which holds the elements in increasing order and d.   (non-ve int.)
// O/P: subsets whose summation is d.
x[k] = 1   // Ist ele with index k
if (W[k] + s == d)
```

```
write (x[1..n])
else if (s+w[k]+w[k+1]<=d)
    sumofsub(s+w[k], k+1, r-w[k])
if (s+r-w[k] >= d) and (s+w[k+1] <= d)
    x[k] = 0
    sumofsub(s, k+1, r-w[k])
```

Branch and Bound: → TSP, → A.P

(We use DFS & BFS so, in this we traverse in both directions)

Assignment Problem:

⇒ Objective is to assign 'n' jobs to 'n' persons such that the total cost of the assignment is as small as possible.

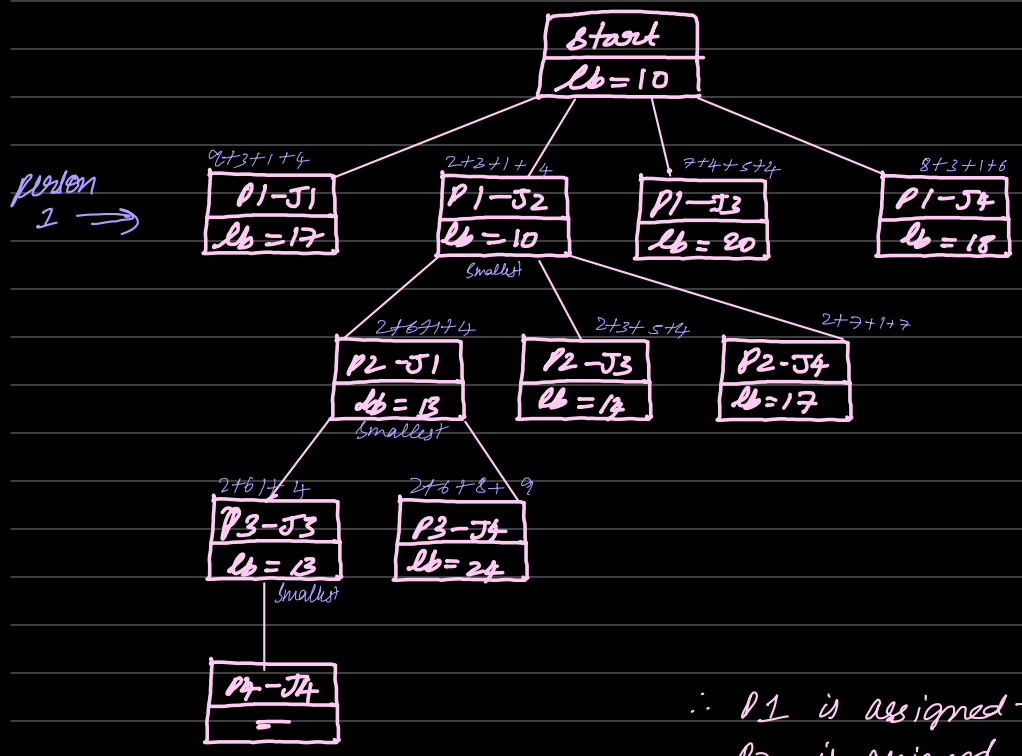
⇒ The A.P is a lower bound problem.

⇒ L.B is cal as the sum of least val in each row.

ex:

	J_1	J_2	J_3	J_4
P_1	9	(2)	7	8
P_2	(6)	4	3	7
P_3	5	8	(1)	8
P_4	7	6	9	(4)

$lb = 2+3+1+4$
$= 10$ //



P1 ⇒ add the elt in Job and all the lower bounds under it. Make sure it is not in the same column as the Job.
ex: 8+3+1+6 ⇒ we do not take 6 as it is in same col as J4

P2 ⇒ 2 is added (of P1) and the elt of job is added along with lower bounds below it. (not in same col as well as we do not consider it if the prev Job already has it
ex: P2-J4 ⇒ 2+7+1+7
⇒ we do not take 7 as it is in same col
⇒ we do not take 6 as it belongs to J2-P1
⇒ we take 3rd smallest ie 7.

P3 ⇒ 2 and 6 are added (of P1 & P2) with other lower bounds. Same rules apply here.

∴ P1 is assigned to J2 with cost = 2
P2 is assigned to J1 with cost = 6
P3 is assigned to J3 with cost = 1
P4 is assigned to J4 with cost = 4
Tot cost ⇒ 13

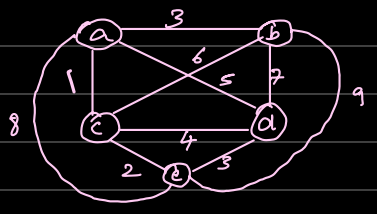
Travelling Salesman problem:

⇒ The main objective of the TSP is to find a route such that a salesperson visits all the cities and returns back to the same city from where he started the tour with the min cost.

⇒ TSP is a minimization problem, hence we are supposed to find the lower bound

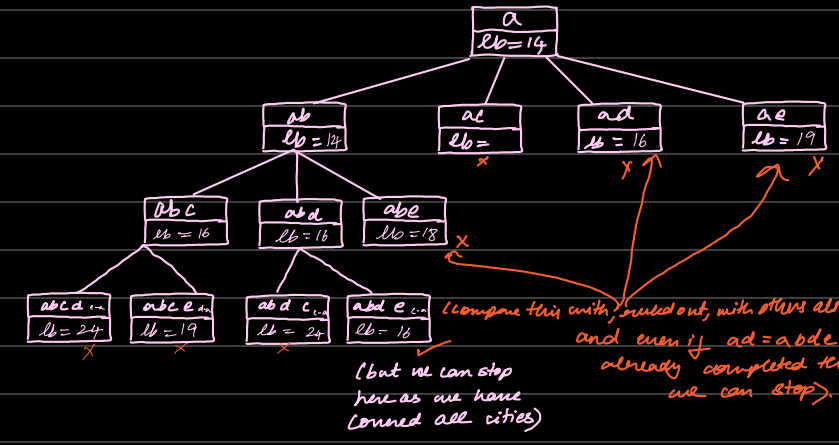
⇒ L.B is $\lceil \frac{s}{2} \rceil$ where s ⇒ summation of 2 closest cities of a city for all cities

Eg:



$$lb = \frac{1+3}{\text{wrt a}} + \frac{3+6}{\text{wrt b}} + \frac{1+2}{c} + \frac{3+4}{d} + \frac{2+3}{e} \Big/ 2 = \lceil \frac{28}{2} \rceil = 14$$

⇒ let a be the starting city.
⇒ city b is visited before city c.



∴ a–b–d–e–c–a

cost = 3+7+3+2+1 = 16

for ab:
⇒ we have to keep the common edge b/w (a,b) const
i.e 3 should be there in a and b
and add all lower bounds.
⇒ $\frac{(3)+1+(3)+6+1+2+3+4+2+3}{2} = \lceil \frac{28}{2} \rceil = 14$

for ac: we do not consider this as we are considering city b before c.

for ad: $\frac{(5)+1+3+6+1+2+(3)+2+e}{2} = \lceil \frac{31}{2} \rceil = \lceil 15.5 \rceil = 16$
⇒ 5 is the edge b/w (a,d) so we take it as common.

for ae: $\frac{(8)+1+3+6+1+2+3+4+(8)+2}{2} = \lceil \frac{38}{2} \rceil = 19$
⇒ since 8 is the common edge of (a,e).

for abc: edges (a,d) and (b,c) should be there
$S \Rightarrow \frac{(5)+1+(3)+6+(6)+1+3+4+2+3}{2}$
$= \lceil \frac{32}{2} \rceil \Rightarrow 16$

for abd: consider (a,b) and (b,d)
$S = \frac{(3)+1+(5)+6+(3)+1+2+(3)+2+3}{2}$
$= \lceil \frac{32}{2} \rceil \Rightarrow 16$

for abe: consider (a,b) and (b,e)
$S = \frac{3+1+3+9+1+2+3+4+9+2}{2}$
$= \lceil \frac{35}{2} \rceil = \lceil 17.5 \rceil = 18$

for abcd: consider (ab)(bc)(cd)(de)(ea)
$S = \frac{3+8+3+6+6+4+4+3+8+2}{2}$
$= \lceil \frac{47}{2} \rceil = \lceil 23.5 \rceil = 24$

for abce: consider (ab)(bc)(ce)(ed)(da)
$S = \frac{3+5+2+6+6+2+5+3+2+3}{2}$
$= \lceil \frac{38}{2} \rceil = 19$

for abdc: consider (ab)(bd)(dc)(ce)(ea)
$S = \frac{3+8+3+7+4+2+5+4+2+8}{2}$
$= \lceil \frac{46}{2} \rceil = 24$

for abde: consider (ab)(bd)(de)(ec)(ca)
$S = \frac{3+1+3+5+2+1+7+5+3+2}{2}$
$= \lceil \frac{32}{2} \rceil = 16$

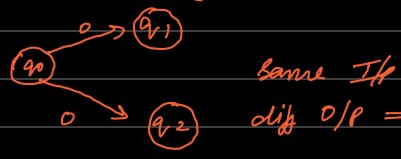
State Space Tree: Is a tree representation where each node represents a state / partial soln to problem and the branches represents the possible choices / decisions that lead to the next state.

⇒ Backtracking involves pruning the branches that do not lead to a valid soln.

N.P hard and N.P Complete problems:

N.P ⇒ Non-Deterministic Alg:

→ O/P cannot be predicted properly even if we know the I/P. Same I/P will give diff outputs for diff rounds of execution.

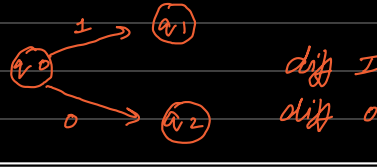
→ 

Same I/P ⇒ 0
diff O/P ⇒ q_1 and q_2.
} non-deterministic alg

→ in non-det algs we cannot determine the next step of execution as the alg will take more than one path (has 2 outputs).
→ we will also get only approximate solns, no accurate solns.

P → Deterministic Algs:

→ In deterministic, for a given I/P the computer will always produce same output, going through the same states

→ 

diff I/P ⇒ 1 & 0.
diff O/P ⇒ q_1 and q_2.
} deterministic alg

---

* P Problem:

→ Problems that can be solved in polynomial time is known as 'P' problem.
→ It can be solved in poly time using deterministic alg. ex: Lin search, Bin search, sort etc
→ significance: problems in P are 'tractable' or efficiently solvable, making them practical for real world applications

** N.P Problem:

→ Problems that cannot be solved in ~~~ are known as 'N.P' problems
→ but they can be verifiable in poly time using non-det algorithm. For example, a prob has to be solved in one hour. If we solve the prob within 1 hour (given polynomial time) then it is a 'P' problem. If not, it is an 'N.P' problem.
→ Significance: crucial as they include many real world problems for which no efficient solv alg is known. The famous "P v/s NP question" asks whether every prob whose soln can be quickly verified (NP) can also be quickly solved (P). This remains one of the most imp open questions in computer science.

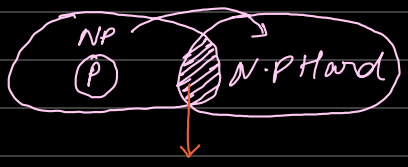
→ depending on computing time the Algs can be classified:

① Polynomial time: algs which takes less time to complete the prob.
ex: linear search (n), binary search ($\log n$), insertion sort (n^2), merge sort ($n \log n$), matrix mult (n^3)//

② Exponential time: algs which takes more time to ~~~ even if it is a small prob.
ex: 0/1 knapsack (2^n), TSP (2^n), sum of subsets (2^n), graph coloring (2^n), hamiltonian cycle (2^n).//

→ Problems that do not need to be solved in poly time can be further classified into 2 types:

① N·P Complete:



→ prob which is 'NP hard' and 'NP' is known as Complete Problem.
→ A problem that is N·P Complete has the property that it can be solved in polynomial time if & only if all other N·P complete problems can be solved in poly time.

② N·P Hard:

A prob is called N·P hard if every problem in in N·P can be polynomially reduced. If an NP hard problem can be solved in polynomial time, then all NP complete problems can be solved in poly time.

Eg: A, B, C, D ⇒ N·P problems. if all these probs can be reduced to 'x' ⇒ N·P hard Problem.

Decision Trees:

→ popular and powerful tool used in various fields such as
* machine learning.
* data mining.
* statistics.

→ They provide a clear, intuitive way to make decisions based on data by modeling the relationships b/w diff variables

→ Structure:

① root node ⇒ represents entire dataset & initial decision.
② internal nodes ⇒ reps decisions / tests.
③ leaf nodes ⇒ reps final decision / prediction.
④ branches ⇒ reps outcome of a decision / test, leading to another node.

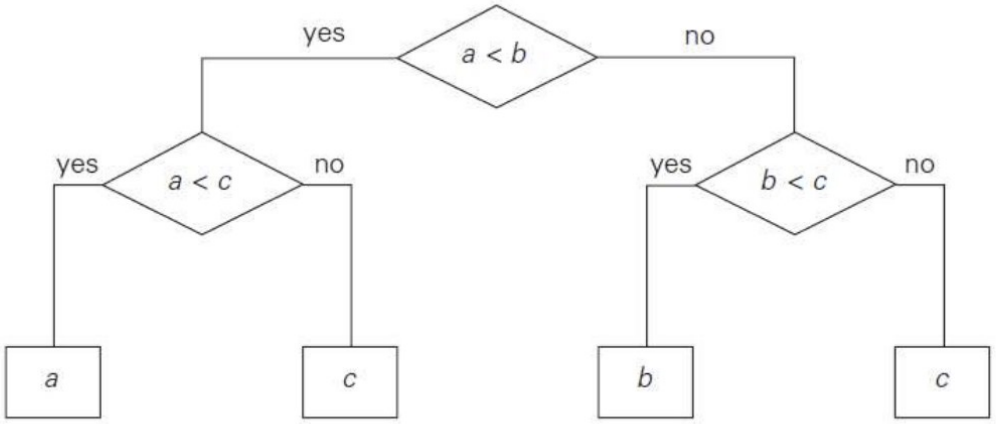
Note:

* no. of comparisions in the worst case is equal to the height of the alg's decision tree.
* $h \geq \lceil \log_2 \ell \rceil$ ⇒ h: height of binary tree
⇒ ℓ: leaves of binary tree.

* Decision trees can be used for analyzing worst & avg-case efficiencies of Comparision-Based sorting Algs.
⇒ $C_{worst}(n) \geq \lceil \log_2 n! \rceil$
⇒ $C_{avg}(n) \geq \log_2 n!$

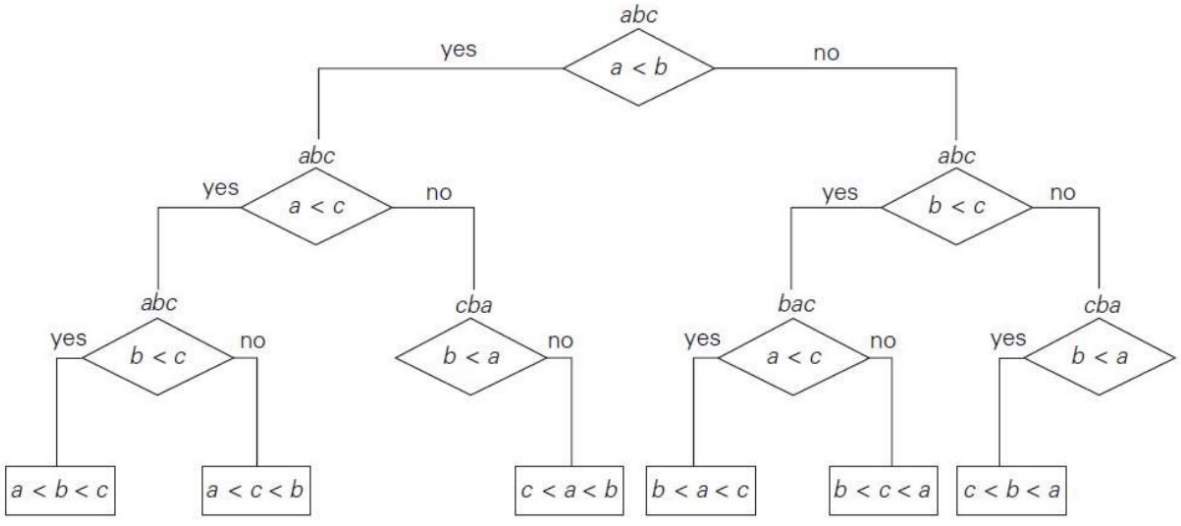
1

Decision tree for finding a minimum of three numbers



2

Decision Trees for Sorting (selection sort)



3

Decision tree for the three element insertion sort